

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN
23TNT1

Đồ án 2

Đề tài: Khai thác tập phổ biến

Môn học: Khai thác dữ liệu và ứng dụng

Sinh viên thực hiện:

22120457 - Khưu Hải Châu

23122006 - Lưu Thượng Hồng

23122020 - Nguyễn Thiên Ân

23122034 - Lê Nguyên Khang

23122036 - Nguyễn Ngọc Khoa

Giáo viên hướng dẫn:

GS.TS. Lê Hoài Bắc

ThS. Lê Nhật Nam

14 tháng 6, 2026



Mục lục

1	Giới thiệu	1
2	Phân công công việc	2
3	Khái niệm và ký hiệu	2
3.1	Bảng ký hiệu	2
3.2	Các khái niệm chính	3
4	Nền tảng lý thuyết	4
4.1	Bài toán Frequent Itemset Mining	4
4.2	Ý tưởng FP-Growth	5
4.3	Cấu trúc dữ liệu FP-tree	5
4.4	Giải mã FP-Growth	7
4.5	Phân tích độ phức tạp	7
4.6	Thuật toán FP-Max	9
4.7	Vị trí lịch sử	10
5	Ví dụ minh họa	10
5.1	Ví dụ 1: CSDL cơ sở	10
5.2	Ví dụ 2: Trường hợp single-path	12
6	Cài đặt	13
6.1	Môi trường và tổ chức mã nguồn	13
6.2	Cài đặt FP-Growth cơ bản	14
6.3	Cài đặt FP-Growth tối ưu	15
6.4	Tránh performance anti-patterns	15
6.5	Cài đặt FP-Max	17
6.6	Kiểm thử tự động	17
6.7	Xử lý đầu vào và đầu ra	18
7	Thực nghiệm và đánh giá	18
7.1	Thiết lập thực nghiệm	18

7.2	Tập dữ liệu benchmark	19
7.3	Tính đúng đắn	19
7.4	Thời gian chạy theo minsup	19
7.5	Số lượng frequent itemset theo minsup	24
7.6	Bộ nhớ	26
7.7	Khả năng mở rộng	27
7.8	Ảnh hưởng của độ dài giao dịch	28
7.9	Nhận xét và hướng tối ưu tiếp theo	29
8	Ứng dụng	30
8.1	Bài toán phân tích giỏ hàng	30
8.2	Dữ liệu và thiết lập	30
8.3	Top-10 luật theo lift	31
8.4	Diễn giải kinh doanh	31
9	Kết luận	32
	Tài liệu	34
A	Phụ lục: Hướng dẫn tái lập	35
A.1	Cấu trúc thư mục	35
A.2	Môi trường và cài đặt	35
A.3	Chạy chương trình	36
A.4	Kiểm thử	36
A.5	Tái lập thực nghiệm Chương 4	37
A.6	Tái lập ứng dụng Chương 5	37
A.7	Tính tái lập	38

Danh sách bảng

1	Bảng phân công công việc của nhóm	2
2	Bảng ký hiệu dùng xuyên suốt báo cáo.	3
3	Độ phức tạp thời gian của FP-Growth theo từng trường hợp.	8
4	Độ phức tạp không gian của FP-Growth theo từng trường hợp.	9
5	CSDL toy dùng để minh hoạ FP-Growth.	10
6	Các giao dịch sau bước tỉa và sắp thứ tự.	11
7	Khai thác conditional pattern base trong ví dụ cơ sở.	11
8	CSDL thiết kế để tạo FP-tree một nhánh.	12
9	Bộ nhớ cấp phát của bản cơ bản và bản tối ưu trên Chess, đo bằng <code>alloc_bytes</code>	17
10	Các tập dữ liệu benchmark dùng trong Mục 7.	19
11	Kết quả correctness đại diện khi so với SPMF.	19
12	Peak RSS thật (<code>sys.maxrss</code>) đo trong tiến trình riêng cho mỗi lần chạy. Cột <i>sàn</i> là cấu hình chỉ load dữ liệu, không mining.	27
13	Ảnh hưởng của độ dài giao dịch trung bình lên thời gian chạy.	28
14	Top-10 luật kết hợp trên Groceries theo lift, với $\text{minsup} = 0.01$ và $\text{minconf} = 0.2$	31

Danh sách hình vẽ

1	FP-tree minh hoạ cho CSDL toy ở Mục 5 với $\text{minsup} = 0.6$	6
2	Thời gian chạy theo minsup trên Chess.	20
3	Thời gian chạy theo minsup trên Mushrooms.	21
4	Thời gian chạy theo minsup trên Retail.	22
5	Thời gian chạy theo minsup trên T10I4D100K.	23
6	Thời gian chạy theo minsup trên Accidents.	24
7	Số frequent itemset theo minsup trên Chess.	25
8	Số frequent itemset theo minsup trên Retail.	25
9	Bộ nhớ cấp phát của base và opt tại minsup trung bình.	26
10	Khả năng mở rộng theo số lượng giao dịch.	28
11	Thời gian chạy khi tăng độ dài giao dịch trung bình.	29

1 Giới thiệu

Khai thác tập phổ biến (Frequent Itemset Mining, FIM) là một bài toán nền tảng trong khai thác dữ liệu giao dịch. Với một cơ sở dữ liệu gồm nhiều giao dịch, mục tiêu của FIM là tìm các nhóm item xuất hiện cùng nhau đủ thường xuyên theo một ngưỡng hỗ trợ tối thiểu. Kết quả của bài toán này thường được dùng làm đầu vào cho luật kết hợp, phân tích giỏ hàng, phát hiện mẫu trong log hệ thống, hoặc phân tích các tổ hợp đặc trưng trong dữ liệu sinh học.

Trong đồ án này, nhóm lựa chọn họ thuật toán FP-Growth và mở rộng thêm phần FP-Max. FP-Growth được đề xuất nhằm tránh bước sinh ứng viên tốn kém của Apriori bằng cách nén cơ sở dữ liệu vào FP-tree và khai thác các cơ sở mẫu điều kiện [Han et al. \(2000\)](#). Trên cơ sở đó, nhóm cài đặt ba thành phần chính: bản FP-Growth cơ bản, bản FP-Growth tối ưu hoá bộ nhớ và tốc độ, và FP-Max để khai thác tập phổ biến tối đại. Cài đặt chính được thực hiện bằng Julia, có giao diện dòng lệnh, bộ kiểm thử tự động, và harness thực nghiệm so sánh với SPMF [Fournier-Viger et al. \(2014\)](#).

Báo cáo được tổ chức như sau. Mục [4](#) trình bày các định nghĩa hình thức của FIM, tính chất Apriori, cấu trúc FP-tree, giả mã FP-Growth và FP-Max, cùng phân tích độ phức tạp. Mục [5](#) minh hoạ thuật toán bằng hai ví dụ tay, gồm một ví dụ cơ sở và một trường hợp đặc biệt single-path. Mục [6](#) mô tả thiết kế cài đặt, các cấu trúc dữ liệu, tối ưu hoá và cách chạy chương trình. Mục [7](#) trình bày thực nghiệm về tính đúng đắn, thời gian chạy, bộ nhớ, khả năng mở rộng và ảnh hưởng của độ dài giao dịch. Mục [8](#) áp dụng thuật toán vào phân tích giỏ hàng trên tập Groceries và sinh luật kết hợp từ các tập phổ biến tìm được.

2 Phân công công việc

Bảng 1: Bảng phân công công việc của nhóm

MSSV	Họ và tên	Công việc
22120457	Khưu Hải Châu	Nghiên cứu lý thuyết FP-Growth, FP-Max và viết Chương 1
23122006	Lưu Thượng Hồng	Cài đặt FP-Growth cơ bản, parser, I/O và CLI Julia
23122020	Nguyễn Thiên Ân	Cài đặt FP-Growth tối ưu, unit test và đối chiếu SPMF
23122034	Lê Nguyên Khang	Thiết kế thực nghiệm, đo thời gian/bộ nhớ và sinh biểu đồ
23122036	Nguyễn Ngọc Khoa	Ứng dụng Groceries, tổng hợp báo cáo và kiểm tra PDF

3 Khái niệm và ký hiệu

Phần này tổng hợp các khái niệm và ký hiệu được dùng nhất quán xuyên suốt báo cáo, nhằm tránh lặp lại định nghĩa ở từng phần. Các định nghĩa hình thức đầy đủ được trình bày ở Mục 4; ở đây chỉ liệt kê để tiện tra cứu. Trừ khi nói rõ là tương đối, mọi giá trị support trong báo cáo đều là support tuyệt đối (đếm số giao dịch).

3.1 Bảng ký hiệu

Bảng 2 liệt kê các ký hiệu toán học chính và ý nghĩa của chúng.

Bảng 2: Bảng ký hiệu dùng xuyên suốt báo cáo.

Ký hiệu	Ý nghĩa
$I = \{i_1, \dots, i_m\}$	Tập tất cả item; m là số item phân biệt
$D = \{T_1, \dots, T_n\}$	Cơ sở dữ liệu giao dịch; $n = D $ là số giao dịch
T_j	Giao dịch thứ j , với $T_j \subseteq I$
X, Y	Itemset, tức tập con của I
$X \subseteq T_j$	Giao dịch T_j chứa itemset X
$\text{sup}(X)$	Support tuyệt đối: số giao dịch trong D chứa X
$\text{sup}_{rel}(X)$	Support tương đối: $\text{sup}(X)/ D $
θ	Ngưỡng support tối thiểu tương đối (minsup)
$\lceil \theta n \rceil$	Ngưỡng support tối thiểu tuyệt đối tương ứng
$X \Rightarrow Y$	Luật kết hợp, với $X \cap Y = \emptyset$
$\text{conf}(X \Rightarrow Y)$	Confidence: $\text{sup}(X \cup Y) / \text{sup}(X)$
$\text{lift}(X \Rightarrow Y)$	Lift: $\text{conf}(X \Rightarrow Y) / \text{sup}_{rel}(Y)$
minconf	Ngưỡng confidence tối thiểu của luật
$\bar{e}$	Độ dài giao dịch trung bình sau bước tủa item không phổ biến
L	Tổng số item còn lại sau tủa trên toàn bộ giao dịch

3.2 Các khái niệm chính

Các thuật ngữ sau xuất hiện lặp lại trong các chương sau và được hiểu thống nhất như dưới đây:

- **Tập phổ biến (frequent itemset):** itemset X với $\text{sup}_{rel}(X) \geq \theta$, tương đương $\text{sup}(X) \geq \lceil \theta n \rceil$ Agrawal et al. (1993).
- **Tập đóng (closed itemset):** tập phổ biến X không có siêu tập phổ biến nào cùng support.
- **Tập tối đại (maximal itemset):** tập phổ biến X không có siêu tập phổ biến nào; mọi tập tối đại đều là tập đóng.
- **Luật kết hợp (association rule):** suy diễn $X \Rightarrow Y$ được đánh giá qua support, confidence và lift Agrawal et al. (1993).
- **FP-tree:** cây nén tiền tố biểu diễn cô đọng cơ sở dữ liệu giao dịch; mỗi node lưu item, biến đếm *count*, con trỏ cha và tập con Han et al. (2000).
- **Header table:** bảng ánh xạ mỗi item tới danh sách các node mang item đó trong FP-tree.

- **Conditional pattern base:** tập các đường tiền tố kết thúc tại một item, dùng để dựng cây điều kiện.
- **Conditional FP-tree:** FP-tree xây từ conditional pattern base của một hậu tố, dùng trong khai thác đệ quy.
- **Single-path:** trường hợp FP-tree (hoặc cây điều kiện) chỉ có một đường đi, cho phép sinh trực tiếp mọi tổ hợp con.
- **MFI (maximal frequent itemset list):** danh sách các tập phổ biến tối đại được FP-Max duy trì trong quá trình khai thác [Grahne and Zhu \(2003\)](#).

4 Nền tảng lý thuyết

4.1 Bài toán Frequent Itemset Mining

Gọi $I = \{i_1, i_2, \dots, i_m\}$ là tập tất cả item. Một cơ sở dữ liệu giao dịch được ký hiệu là $D = \{T_1, T_2, \dots, T_n\}$, trong đó mỗi giao dịch $T_j \subseteq I$. Một itemset X là một tập con của I . Ta nói giao dịch T_j chứa X nếu $X \subseteq T_j$.

Độ hỗ trợ tuyệt đối của X trên D là số giao dịch chứa X :

$$\text{sup}(X) = |\{T_j \in D \mid X \subseteq T_j\}|.$$

Độ hỗ trợ tương đối là $\text{sup}_{rel}(X) = \text{sup}(X)/|D|$. Với ngưỡng hỗ trợ tối thiểu θ , itemset X được gọi là phổ biến nếu $\text{sup}_{rel}(X) \geq \theta$, hay tương đương $\text{sup}(X) \geq \lceil \theta |D| \rceil$.

Bên cạnh tập phổ biến thông thường, hai biến thể quan trọng là tập đóng và tập tối đại. Một itemset phổ biến X là *closed itemset* nếu không tồn tại itemset phổ biến Y sao cho $X \subset Y$ và $\text{sup}(X) = \text{sup}(Y)$. Một itemset phổ biến X là *maximal itemset* nếu không tồn tại itemset phổ biến Y sao cho $X \subset Y$. Do đó mọi maximal itemset đều là closed itemset, nhưng không phải mọi closed itemset đều là maximal itemset.

Tính chất quan trọng nhất của FIM là tính phản đơn điệu của support, thường được gọi là tính chất Apriori [Agrawal et al. \(1993\)](#). Nếu $X \subseteq Y$ thì mọi giao dịch chứa Y cũng chứa X , nên:

$$\text{sup}(Y) \leq \text{sup}(X).$$

Hệ quả là nếu X không phổ biến thì mọi siêu tập của X cũng không phổ biến. Ngược lại, nếu Y phổ biến thì mọi tập con của Y cũng phổ biến. Tính chất này cho phép các thuật toán FIM tìm kiếm không gian tìm kiếm rất mạnh.

4.2 Ý tưởng FP-Growth

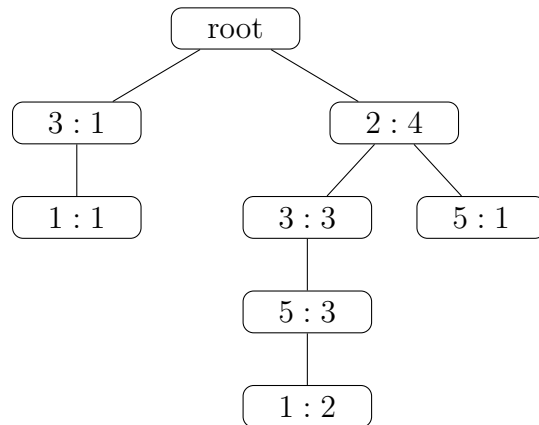
Apriori khai thác tính chất phản đơn điệu bằng cách sinh ứng viên theo từng mức độ dài Agrawal and Srikant (1994). Cách làm này dễ hiểu nhưng phải quét cơ sở dữ liệu nhiều lần và có thể sinh rất nhiều ứng viên trung gian. FP-Growth thay đổi hướng tiếp cận: thuật toán chỉ quét dữ liệu hai lần để xây dựng một cây nén gọi là FP-tree, sau đó khai thác mẫu bằng các cơ sở mẫu điều kiện mà không cần sinh ứng viên toàn cục Han et al. (2000).

Trực giác của FP-Growth là nhiều giao dịch có tiền tố giống nhau sau khi các item được sắp theo support giảm dần. Khi chèn các giao dịch đã sắp xếp vào cùng một cây, các tiền tố chung được gộp lại và đếm bằng biến *count* trên node. Header table lưu danh sách các node ứng với cùng một item, nhờ đó thuật toán có thể tìm nhanh các đường đi kết thúc tại item đang xét. Với mỗi item, FP-Growth thu thập conditional pattern base, dựng conditional FP-tree, rồi đệ quy khai thác các mẫu có hậu tố là item đó.

4.3 Cấu trúc dữ liệu FP-tree

Một node trong FP-tree gồm bốn thành phần: item đang đại diện, biến đếm số giao dịch đi qua node, con trỏ tới node cha, và tập các node con. Root không chứa item thật. Header table ánh xạ mỗi item tới danh sách các node mang item đó trong cây.

Ví dụ với cơ sở dữ liệu toy ở Mục 5 và $\theta = 0.6$, các item phổ biến là 2, 3, 5, 1 với support lần lượt 4, 4, 4, 3. Sau khi loại item không phổ biến và sắp item theo support giảm dần, FP-tree có dạng khái niệm trong Hình 1.



Hình 1: FP-tree minh hoạ cho CSDL toy ở Mục 5 với minsup = 0.6.

Cây trên nén năm giao dịch thành sáu node item. Hai giao dịch $[2, 3, 5, 1]$ trùng tiền tố hoàn toàn nên được biểu diễn bằng một đường đi có count tăng lên. Đây là nguồn tiết kiệm bộ nhớ chính của FP-Growth trên dữ liệu có nhiều tiền tố chung.

4.4 Giải mã FP-Growth

Algorithm 1 FP-Growth

Require: Cơ sở dữ liệu giao dịch D , ngưỡng θ

Ensure: Tập frequent itemset và support tương ứng

```
1: Đếm support của từng item trong  $D$ 
2: Loại các item có support nhỏ hơn  $\lceil \theta |D| \rceil$ 
3: Sắp item trong mỗi giao dịch theo support giảm dần, nếu bằng nhau thì theo mã item
4: Chèn các giao dịch đã sắp vào FP-tree và cập nhật header table
5: Gọi  $\text{MINE}(\text{tree}, \emptyset)$ 
6: function  $\text{MINE}(\text{tree}, \text{suffix})$ 
7:   if  $\text{tree}$  chỉ có một đường đi then
8:     Sinh mọi tập con không rỗng của đường đi
9:     Ghi nhận mỗi tập con hợp với  $\text{suffix}$ , support là count nhỏ nhất trên tập con
10:    return
11:  end if
12:  for mỗi item  $a$  trong header table do
13:     $\text{pattern} \leftarrow \text{suffix} \cup \{a\}$ 
14:    Ghi nhận  $\text{pattern}$  với support của  $a$  trong  $\text{tree}$ 
15:    Thu thập conditional pattern base của  $a$ 
16:    Loại item không đủ support trong conditional pattern base
17:    Dựng conditional FP-tree
18:    if conditional FP-tree không rỗng then
19:       $\text{MINE}(\text{conditional FP-tree}, \text{pattern})$ 
20:    end if
21:  end for
22: end function
```

Điểm mấu chốt trong giải mã là thuật toán không sinh ứng viên toàn cục như Apriori. Mỗi lần đệ quy chỉ làm việc trên phần dữ liệu điều kiện liên quan tới hậu tố đang xét. Khi cây điều kiện chỉ còn một đường đi, thuật toán có thể sinh trực tiếp các tổ hợp trên đường đi đó, tránh dựng thêm cây không cần thiết.

4.5 Phân tích độ phức tạp

Độ phức tạp thời gian. Chi phí gồm hai giai đoạn: xây FP-tree và khai thác đệ quy. Giai đoạn xây cây cần hai lần quét CSDL — một để đếm support, một để chèn giao dịch đã sắp — với chi phí $\mathcal{O}(L \log \bar{\ell})$ do mỗi giao dịch phải sắp lại theo support ($\bar{\ell}$ là độ dài giao dịch trung bình sau tĩa). Giai đoạn khai thác phụ thuộc số frequent itemset F và chi phí dựng, duyệt các conditional FP-tree.

- **Tốt nhất (single-path):** cây co thành một đường, mọi tổ hợp con sinh trực tiếp không cần

dựng conditional tree. Khai thác tỉ lệ với số itemset xuất ra, tổng $\mathcal{O}(L \log \bar{\ell} + F)$.

- **Trung bình:** trên dữ liệu thực F nhỏ hơn rất nhiều cận 2^m , thời gian khai thác xấp xỉ tuyến tính theo F nhân chi phí trung bình mỗi conditional tree. Tổng $\mathcal{O}(L \log \bar{\ell} + F \cdot c)$ với c là chi phí trung bình dựng/duyệt một cây điều kiện.
- **Xấu nhất:** mọi tập con của m item phổ biến đều đạt ngưỡng, $F = 2^m - 1$. Chi phí khai thác bùng nổ mũ $\mathcal{O}(2^m)$, chi phối toàn bộ thời gian chạy.

Bảng 3: Độ phức tạp thời gian của FP-Growth theo từng trường hợp.

Trường hợp	Xây cây	Khai thác
Tốt nhất	$\mathcal{O}(L \log \bar{\ell})$	$\mathcal{O}(F)$
Trung bình	$\mathcal{O}(L \log \bar{\ell})$	$\mathcal{O}(F \cdot c)$
Xấu nhất	$\mathcal{O}(L \log \bar{\ell})$	$\mathcal{O}(2^m)$

Kết quả này nhất quán với thực nghiệm: thời gian chạy tăng theo số itemset đầu ra (Mục 7), và bước nhảy mạnh khi output tăng từ 100 lên 5,050 itemset trong thí nghiệm độ dài giao dịch (Bảng 13) đúng với cận mũ ở trường hợp xấu.

Độ phức tạp không gian. Bộ nhớ FP-Growth gồm FP-tree cùng header table, các conditional FP-tree sinh trong đệ quy, và tập kết quả. Gọi N là số node thực tế của FP-tree và F số frequent itemset.

- **Tốt nhất (single-path):** mọi giao dịch chung tiền tố, cây co thành một đường dài $\mathcal{O}(\bar{\ell})$ và không sinh conditional tree. Bộ nhớ $\mathcal{O}(\bar{\ell} + F)$, độc lập với n — đây là lợi ích của tối ưu single-path.
- **Trung bình:** nén tiền tố tốt nên $N \ll L$; các conditional tree dọc một nhánh được giải phóng khi nhánh kết thúc, độ sâu đệ quy $\mathcal{O}(\bar{\ell})$. Bộ nhớ làm việc xấp xỉ $\mathcal{O}(N + F)$, khớp với Bảng 12: peak RSS chỉ vượt sần vài chục MB.
- **Xấu nhất:** không có tiền tố chung, cây không nén được nên $N = \mathcal{O}(L)$; nếu mọi tập con đều phổ biến thì $F = 2^m - 1$, output chiếm ưu thế. Tổng $\mathcal{O}(L + 2^m)$.

Bảng 4: Độ phức tạp không gian của FP-Growth theo từng trường hợp.

Trường hợp	Cấu trúc	Output	Tổng
Tốt nhất	$\mathcal{O}(\bar{\ell})$	$\mathcal{O}(F)$	$\mathcal{O}(\bar{\ell} + F)$
Trung bình	$\mathcal{O}(N), N \ll L$	$\mathcal{O}(F)$	$\mathcal{O}(N + F)$
Xấu nhất	$\mathcal{O}(L)$	$\mathcal{O}(2^m)$	$\mathcal{O}(L + 2^m)$

Cận trên ở đây là *đỉnh thường trú* (Sys.maxrss), khác với tổng alloc_bytes (Bảng 9) vốn cộng dồn cả các Vector tạm bị GC thu hồi — nên tổng cấp phát thường lớn hơn đỉnh thường trú nhiều lần.

4.6 Thuật toán FP-Max

FP-Max khai thác tập phổ biến tối đại thay vì toàn bộ tập phổ biến [Grahne and Zhu \(2003\)](#). Một itemset phổ biến là tối đại nếu không có siêu tập phổ biến nào chứa nó. Ý tưởng của FP-Max là dùng cùng cơ chế FP-tree/conditional pattern base như FP-Growth, nhưng duy trì một danh sách các maximal frequent itemsets đã tìm được (MFI list). Khi xét một head hiện tại và tail còn có thể mở rộng, nếu $head \cup tail$ đã là tập con của một MFI có sẵn thì toàn bộ nhánh đó có thể bị tĩa.

Algorithm 2 FP-Max

Require: FP-tree, head hiện tại, danh sách MFI

Ensure: Danh sách maximal frequent itemsets

```

1: if FP-tree chỉ có một đường đi then
2:    $candidate \leftarrow head \cup$  các item trên đường đi
3:   Chèn  $candidate$  vào MFI nếu nó không là tập con của MFI hiện có
4:   Xoá các MFI cũ là tập con nghiêm ngặt của  $candidate$ 
5:   return
6: end if
7: Sắp các item trong header theo support tăng dần
8: for mỗi item  $a$  do
9:    $head \leftarrow head \cup \{a\}$ 
10:  Tạo conditional pattern base và tail phổ biến của  $a$ 
11:  if  $head \cup tail$  không là tập con của MFI nào then
12:    Dựng conditional FP-tree và đệ quy
13:  end if
14:  Bỏ  $a$  khỏi head
15: end for

```

FP-Max hữu ích khi người dùng chỉ cần biểu diễn cô đọng của các mẫu phổ biến. Từ các maximal itemsets có thể biết vùng biên của không gian frequent itemset, nhưng không thể khôi phục chính

xác support của mọi tập con nếu không quét lại dữ liệu. Vì vậy trong đồ án này FP-Max được cài đặt như một nhánh bổ sung, còn phần thực nghiệm chính tập trung vào FP-Growth và bản tối ưu.

4.7 Vị trí lịch sử

Apriori đặt nền móng cho FIM bằng tính chất phản đơn điệu, nhưng bị giới hạn bởi số lần quét dữ liệu và số lượng ứng viên lớn [Agrawal and Srikant \(1994\)](#). FP-Growth giải quyết hạn chế này bằng FP-tree và khai thác mẫu không sinh ứng viên. Sau đó, nhiều hướng tối ưu tiếp tục xuất hiện: các thuật toán tập đóng/tối đại để giảm kích thước output, các thuật toán dạng dọc như Eclat dùng tidset/diffset [Zaki \(2000\)](#), và các thư viện thực nghiệm như SPMF cung cấp nhiều cài đặt tham chiếu [Fournier-Viger et al. \(2014\)](#). Trong dòng phát triển này, FP-Growth vẫn là một thuật toán nền tảng vì cân bằng tốt giữa tính dễ hiểu, khả năng nén dữ liệu và hiệu quả thực nghiệm trên nhiều loại dữ liệu giao dịch.

5 Ví dụ minh họa

5.1 Ví dụ 1: CSDL cơ sở

Xét cơ sở dữ liệu D gồm 5 giao dịch trong Bảng 5. Nhóm chọn minsup tương đối $\theta = 0.6$, do đó minsup tuyệt đối là $\lceil 0.6 \times 5 \rceil = 3$.

Bảng 5: CSDL toy dùng để minh họa FP-Growth.

Giao dịch	Item
T_1	$\{1, 3, 4\}$
T_2	$\{2, 3, 5\}$
T_3	$\{1, 2, 3, 5\}$
T_4	$\{2, 5\}$
T_5	$\{1, 2, 3, 5\}$

Bước đầu tiên là đếm support của từng item. Kết quả là $\text{sup}(1) = 3$, $\text{sup}(2) = 4$, $\text{sup}(3) = 4$, $\text{sup}(4) = 1$, $\text{sup}(5) = 4$. Vì minsup tuyệt đối bằng 3, item 4 bị loại. Các item còn lại được sắp theo support giảm dần, nếu bằng nhau thì theo mã item tăng dần: $2 \prec 3 \prec 5 \prec 1$.

Bảng 6: Các giao dịch sau bước tĩa và sắp thứ tự.

Giao dịch	Sau khi loại item không phổ biến	Sau khi sắp theo thứ tự FP-tree
T_1	$\{1, 3\}$	$[3, 1]$
T_2	$\{2, 3, 5\}$	$[2, 3, 5]$
T_3	$\{1, 2, 3, 5\}$	$[2, 3, 5, 1]$
T_4	$\{2, 5\}$	$[2, 5]$
T_5	$\{1, 2, 3, 5\}$	$[2, 3, 5, 1]$

Chèn tuần tự các giao dịch trong Bảng 6 tạo ra FP-tree đã minh hoạ ở Hình 1. Từ cây này, thuật toán khai thác từng item thông qua conditional pattern base; kết quả từng bước được trình bày trong Bảng 7:

Bảng 7: Khai thác conditional pattern base trong ví dụ cơ sở.

Item hậu tố	Conditional pattern base	Itemset phổ biến sinh ra
1	$(3) : 1, (2, 3, 5) : 2$	$\{1\} : 3, \{1, 3\} : 3$
5	$(2, 3) : 3, (2) : 1$	$\{5\} : 4, \{2, 5\} : 4, \{3, 5\} : 3, \{2, 3, 5\} : 3$
3	$\emptyset : 1, (2) : 3$	$\{3\} : 4, \{2, 3\} : 3$
2	$\emptyset : 4$	$\{2\} : 4$

Tập kết quả cuối cùng gồm:

$$\begin{aligned} &\{1\} : 3, \{2\} : 4, \{3\} : 4, \{5\} : 4, \\ &\{1, 3\} : 3, \{2, 3\} : 3, \{2, 5\} : 4, \{3, 5\} : 3, \\ &\{2, 3, 5\} : 3. \end{aligned}$$

Để kiểm tra chéo thủ công, ta liệt kê các itemset từ các item phổ biến $\{1, 2, 3, 5\}$. Các cặp có support đạt ngưỡng là $\{1, 3\}$, $\{2, 3\}$, $\{2, 5\}$ và $\{3, 5\}$. Trong các bộ ba, chỉ $\{2, 3, 5\}$ xuất hiện trong T_2, T_3, T_5 nên đạt support 3. Không có bộ bốn nào đạt ngưỡng vì $\{1, 2, 3, 5\}$ chỉ xuất hiện trong hai giao dịch. Kết quả này trùng với kết quả từ FP-Growth.

Nếu xét tập phổ biến tối đại, hai itemset tối đại là $\{1, 3\} : 3$ và $\{2, 3, 5\} : 3$. Các itemset còn lại đều là tập con của ít nhất một trong hai itemset này.

5.2 Ví dụ 2: Trường hợp single-path

Trường hợp đặc biệt quan trọng của FP-Growth là FP-tree chỉ có một đường đi. Khi đó thuật toán không cần dựng tiếp conditional tree; mọi frequent itemset có thể sinh trực tiếp từ các tổ hợp con trên đường đi.

Xét CSDL trong Bảng 8 với $\theta = 0.6$, minsup tuyệt đối bằng 3.

Bảng 8: CSDL thiết kế để tạo FP-tree một nhánh.

Giao dịch	Item
T_1	$\{A, B, C, D\}$
T_2	$\{A, B, C\}$
T_3	$\{A, B\}$
T_4	$\{A, B, C\}$
T_5	$\{A, B, C, D\}$

Support của các item là $\text{sup}(A) = 5$, $\text{sup}(B) = 5$, $\text{sup}(C) = 4$, $\text{sup}(D) = 2$. Item D bị loại vì support nhỏ hơn 3. Sau khi tĩa, mọi giao dịch đều có dạng tiền tố của $[A, B, C]$, nên FP-tree chỉ còn một đường:

$$\text{root} \rightarrow A : 5 \rightarrow B : 5 \rightarrow C : 4.$$

Với single-path, mọi tập con không rỗng của $\{A, B, C\}$ đều phổ biến. Support của một itemset bằng count nhỏ nhất trên các node thuộc itemset đó:

$$\begin{aligned} \{A\} : 5, \{B\} : 5, \{C\} : 4, \\ \{A, B\} : 5, \{A, C\} : 4, \{B, C\} : 4, \\ \{A, B, C\} : 4. \end{aligned}$$

Trường hợp này đặc biệt vì conditional tree càng sâu thường càng nhỏ và dễ trở thành single-path. Tối ưu single-path giúp FP-Growth chuyển một nhánh đệ quy thành bài toán sinh tổ hợp trực tiếp trên một đường đi ngắn, giảm đáng kể chi phí khi dữ liệu có cấu trúc tiền tố lồng nhau.

6 Cài đặt

6.1 Môi trường và tổ chức mã nguồn

Cài đặt chính của nhóm sử dụng Julia theo đúng ưu tiên của đề bài. Mã nguồn được tổ chức như một package tên `FrequentItemsetMining`, trong đó file `src/FrequentItemsetMining.jl` là module entry và include các file con. Bố cục chính của repo như sau:

- `src/structures.jl`: định nghĩa `Transaction`, `DataLoader`, `FPNode`, `FPNodeOpt`.
- `src/utils.jl`: tiện ích chung, ví dụ hàm `combinations` sinh tổ hợp dùng trong khai thác và sinh luật.
- `src/data_loader.jl`: đọc dữ liệu giao dịch, hỗ trợ dòng item cách nhau bằng khoảng trắng theo định dạng SPMF và dòng có dấu phẩy.
- `src/algorithm/fp_growth.jl`: cài đặt FP-Growth cơ bản.
- `src/algorithm/fp_growth_opt.jl`: cài đặt FP-Growth tối ưu.
- `src/algorithm/fp_max.jl`: cài đặt FP-Max.
- `src/association_rules.jl`: định nghĩa `AssociationRule` và `generate_rules` để sinh luật kết hợp từ tập phổ biến.
- `src/io.jl`: xuất itemsets theo dạng `item item ... #SUP: count`.
- `main.jl`: giao diện dòng lệnh.
- `test/`: bộ unit test correctness và benchmark smoke test.
- `experiments/`: script đối chiếu SPMF, sinh CSV và sinh hình cho Mục 7.

Trước khi chuyển sang Julia, nhóm có một bản phác thảo (prototype) bằng Python đặt trong thư mục `python/`. Bản này chỉ dùng để kiểm chứng ý tưởng ban đầu của FP-Growth và FP-Max; toàn bộ kết quả, thực nghiệm và đánh giá trình bày trong báo cáo đều dựa trên cài đặt Julia.

Lệnh chạy chương trình có dạng:

```
1 julia --project=. main.jl <input_path> <minsup> [fp-growth|fp-growth-opt|fp-max|  
  rules] [output_path]
```

Listing 1: Cách chạy chương trình từ dòng lệnh.

Ví dụ, để khai thác tập phổ biến trên CSDL toy với ngưỡng 0.6:

```
1 julia --project=. main.jl data/toy/test_1.txt 0.6 fp-growth-opt output.txt
```

Listing 2: Ví dụ chạy FP-Growth tối ưu.

Ngoài ba thuật toán khai thác itemset, CLI còn có chế độ **rules** để sinh trực tiếp luật kết hợp từ tập phổ biến (dùng FP-Growth tối ưu), lọc theo lift > 1 và sắp theo lift giảm dần; ngưỡng confidence đặt qua biến môi trường MINCONF (mặc định 0.2). Với dữ liệu có item chứa khoảng trắng trong tên như Groceries, đặt SEP=comma để parser chỉ tách theo dấu phẩy:

```
1 SEP=comma julia --project=. main.jl data/groceries.txt 0.01 rules rules.csv
```

Listing 3: Sinh luật kết hợp trên Groceries qua CLI.

Lệnh này tái tạo đúng kết quả của phần ứng dụng (Mục 8): 333 frequent itemsets và 231 luật thỏa minconf với lift lớn hơn 1.

6.2 Cài đặt FP-Growth cơ bản

Bản cơ bản dùng node lưu item dạng **String**. Mỗi node có **count**, con trỏ **parent** và tập **children**. Khi chèn item vào một node cha, chương trình duyệt các con hiện có để tìm child cùng item; nếu có thì tăng count, nếu không thì tạo node mới. Sau khi xây cây, hàm **rebuild_nodes_table!** duyệt cây để dựng header table.

Quy trình **fit!** của **FPGrowth** gồm bốn bước. Thứ nhất, gom toàn bộ giao dịch và tính minsup tuyệt đối bằng $\lceil \theta n \rceil$. Thứ hai, đếm support từng item và loại item không đủ ngưỡng. Thứ ba, trong mỗi giao dịch, giữ lại item phổ biến và sắp theo **(-support, item)** để kết quả có thứ tự ổn định. Cuối cùng, chèn các giao dịch vào FP-tree.

Hàm **get_frequent_itemsets** khai thác từng item bằng **mine_roads**. Với mỗi node của item đang xét, hàm đi ngược theo con trỏ cha để lấy đường tiền tố. Các đường tiền tố tạo thành conditional pattern base. Bản cơ bản sau đó đếm các item hợp lệ trong base, sinh các tổ hợp con của chúng và tính support bằng cách kiểm tra đường nào chứa tổ hợp đó. Cách này dễ kiểm chứng và phù hợp làm baseline, nhưng có thể tốn kém khi conditional base rộng vì phải enumerate nhiều tổ hợp.

6.3 Cài đặt FP-Growth tối ưu

Bản `FPGrowthOpt` giữ cùng API với bản cơ bản nhưng thay đổi cấu trúc dữ liệu và cách khai thác. Các item được mã hoá từ `String` sang `Int`. Mỗi node tối ưu lưu `item::Int`, `count::Int`, `parent`, và `children::Dict{Int, FPNodeOpt}`. Nhờ vậy thao tác tìm child khi chèn path có chi phí trung bình $O(1)$ thay vì duyệt tuyến tính qua tập con.

Các kỹ thuật tối ưu chính gồm:

- **Integer encoding**: khai thác trên mã số nguyên và chỉ giải mã về chuỗi ở bước cuối.
- **Header table typed**: header ánh xạ `Int` sang danh sách node, giảm overhead so với item chuỗi.
- **Recursive conditional FP-tree**: khai thác đúng hướng FP-Growth chuẩn, tránh enumerate tổ hợp trên conditional base rộng như bản cơ bản.
- **Single-path shortcut**: nếu cây điều kiện chỉ có một nhánh, sinh trực tiếp mọi tổ hợp con của nhánh đó.
- **Tỉa sớm**: trong mỗi conditional pattern base, item có support nhỏ hơn minsup tuyệt đối bị loại trước khi dựng cây điều kiện.
- **Type-stable hot loop**: các trường dữ liệu được khai báo kiểu cụ thể và một số vòng lặp dùng `@inbounds` khi truy cập vector.

Output của bản tối ưu vẫn là `Dict{Tuple{Vararg{String}}, Int}`, giống bản cơ bản. Điều này giúp unit test có thể so sánh trực tiếp:

```
get_frequent_itemsets(FPGrowthOpt) == get_frequent_itemsets(FPGrowth)
```

6.4 Tránh performance anti-patterns

Ở bản tối ưu, nhóm không chỉ thêm các macro hiệu năng vào bản cơ bản mà tổ chức lại dữ liệu để trình biên dịch Julia có đủ thông tin về kiểu trong các đoạn chạy nhiều nhất. Trạng thái của thuật toán được gom trong `FPGrowthOpt`: ngưỡng support, bảng mã hoá item, root của FP-tree và header table đều là các trường có kiểu cụ thể. Vì vậy quá trình dựng cây và khai thác không phụ

thuộc vào biến toàn cục thay đổi được, tránh tình huống Julia phải suy luận kiểu lại qua từng lần truy cập.

Các node trên FP-tree cũng được thiết kế theo hướng này. `FPNodeOpt` lưu item dưới dạng `Int`, `count` dưới dạng `Int`, và tập con bằng `Dict{Int, FPNodeOpt}`. Header table tương ứng có kiểu `Dict{Int, Vector{FPNodeOpt}}`. Những cấu trúc này giúp thao tác chèn path và tính support đi qua các container có kiểu rõ ràng, thay vì làm việc với item chuỗi hoặc container abstract trong hot loop. Trường `parent::Union{Nothing, FPNodeOpt}` vẫn được giữ vì cần cho bước dựng conditional pattern base; đây là một union nhỏ và không làm thay đổi kiểu của node hay header table.

Các macro tối ưu được dùng có chọn lọc. `@inbounds` chỉ xuất hiện ở những vòng lặp mà biên truy cập đã được đảm bảo bởi chính cấu trúc vòng lặp, như duyệt path trong `insert_path!`, cộng support trong `_item_support`, và sinh tổ hợp trên single-path trong `_mine!`. Ngược lại, `@simd` không được dùng vì phần tốn thời gian của FP-Growth chủ yếu là duyệt cây và tra cứu `Dict`. Đây là dạng truy cập rời rạc, phụ thuộc con trỏ và nhánh điều kiện; ép vector hoá ở đây không phù hợp với bản chất của thuật toán.

Nhóm kiểm chứng tính ổn định kiểu bằng `@code_warntype` và `@inferred` trên các hàm nóng. Với `_item_support`, `@code_warntype` cho thấy mọi biến cục bộ và kiểu trả về đều cụ thể:

Arguments

```
nodes::Vector{FPNodeOpt}
```

Locals

```
s::Int64
```

```
nd::FPNodeOpt
```

```
Body::Int64
```

Không có biến nào bị suy ra kiểu `Any`; union duy nhất xuất hiện là trạng thái iterator chuẩn của vòng lặp `for` trong Julia, không phải bất ổn kiểu. Lệnh `@inferred FrequentItemsetMining._item_support(nodes)` chạy không lỗi và suy ra `Int64`, xác nhận hàm nóng này type-stable.

Lợi ích cấp phát cũng được đo trực tiếp thay vì suy luận. Bảng 9 so sánh lượng bộ nhớ cấp phát của bản cơ bản và bản tối ưu trên Chess ở ba ngưỡng minsup (cột `alloc_bytes` từ `timing.csv`). Khoảng cách tăng nhanh khi minsup giảm và output lớn hơn: từ khoảng 1.32 lần ở minsup 0.9 lên 11.33 lần ở minsup 0.8. Điều này cho thấy việc giữ kiểu cụ thể và tránh enumerate tổ hợp dư thừa có tác động đo được ở đúng vùng mà chi phí cấp phát chiếm ưu thế.

Bảng 9: Bộ nhớ cấp phát của bản cơ bản và bản tối ưu trên Chess, đo bằng `alloc_bytes`.

minsup	base (MB)	opt (MB)	Tỉ lệ giảm
0.80	360.78	31.84	11.33
0.85	48.29	18.19	2.65
0.90	17.35	13.17	1.32

6.5 Cài đặt FP-Max

FPMax sử dụng `FPGrowth` để dựng cây ban đầu, đồng thời lưu lại toàn bộ giao dịch dưới dạng `Set{String}` để tính support của maximal itemsets ở bước cuối. Hàm `get_maximal_itemsets` duy trì danh sách các MFI dưới dạng vector các set. Khi có candidate mới, hàm `insert_mfi!` bỏ qua candidate nếu nó là tập con của một MFI có sẵn; ngược lại, hàm xoá các MFI cũ là tập con nghiêm ngặt của candidate rồi thêm candidate mới.

Trong đệ quy FP-Max, nếu cây là single-path thì thuật toán hợp head hiện tại với toàn bộ item trên đường đi và cập nhật MFI. Nếu cây có nhiều nhánh, thuật toán xét các item trong header theo support tăng dần, tạo conditional pattern base, tìm tail còn phổ biến, rồi dùng kiểm tra $head \cup tail$ để tĩa các nhánh chắc chắn không sinh MFI mới.

6.6 Kiểm thử tự động

Bộ kiểm thử trong `test/test_correctness.jl` gồm ba nhóm chính. Nhóm đầu kiểm tra parser với ba định dạng: `1, 2, 3`, `{1, 2, 3}`, và `1 2 3`. Nhóm thứ hai kiểm tra FP-Growth cơ bản trên CSDL toy ở Mục 5, kỳ vọng đúng 9 frequent itemsets với support tương ứng. Nhóm thứ ba kiểm tra FP-Max trên cùng CSDL, kỳ vọng hai maximal itemsets `{1, 3}` và `{2, 3, 5}`.

Ngoài ra, test đối chiếu `FPGrowthOpt` với `FPGrowth` trên `data/toy/test_1.txt`, `data/toy/test_2.txt` và `data/benchmark/chess.txt` ở nhiều giá trị minsup. Test benchmark trong `test/test_benchmark.jl` đo thời gian và số byte cấp phát trên `chess` với minsup 0.9, sau đó kiểm tra lại kết quả của bản tối ưu và bản cơ bản phải trùng nhau. Lệnh kiểm thử chuẩn là:

```
1 julia --project=. test/runtests.jl
```

Listing 4: Lệnh chạy bộ test.

6.7 Xử lý đầu vào và đầu ra

Input chuẩn của đồ án là mỗi giao dịch trên một dòng, item cách nhau bằng khoảng trắng theo định dạng SPMF. Parser cũng hỗ trợ dòng có dấu phẩy để thuận tiện cho dữ liệu giỏ hàng. Output của `write_itemsets` sắp các itemset theo độ dài và thứ tự từ điển, sau đó ghi mỗi dòng theo dạng:

```
1 item1 item2 ... #SUP: support_count
```

Listing 5: Định dạng output itemset.

Định dạng này tương thích với output của SPMF, giúp script thực nghiệm đọc kết quả của hai phía và so sánh từng itemset theo support.

7 Thực nghiệm và đánh giá

7.1 Thiết lập thực nghiệm

Thực nghiệm được thiết kế để kiểm tra sáu khía cạnh: tính đúng đắn so với SPMF, thời gian chạy theo minsup, số lượng frequent itemset theo minsup, bộ nhớ cấp phát, khả năng mở rộng theo số giao dịch, và ảnh hưởng của độ dài giao dịch trung bình. Kết quả được lưu trong các file CSV ở `experiments/results/`; hình minh hoạ được sinh vào `experiments/figures/`.

Các thuật toán được so sánh gồm:

- **base**: bản FP-Growth cơ bản của nhóm.
- **opt**: bản FP-Growth tối ưu của nhóm.
- **spmf**: cài đặt FP-Growth trong thư viện SPMF [Fournier-Viger et al. \(2014\)](#), dùng làm tham chiếu.

Bản cơ bản có chiến lược enumerate tổ hợp trong conditional base nên không chạy ở các cấu hình có nguy cơ bùng nổ bộ nhớ, đặc biệt trên dữ liệu dày hoặc minsup thấp. Các cấu hình này được đánh dấu **skip** trong CSV thay vì ép chạy đến khi tiến trình bị lỗi. Với Julia, thời gian được đo sau một lần warm-up để giảm ảnh hưởng của JIT; lượng cấp phát dùng cột `alloc_bytes` từ `@timed`, còn peak memory thật được đo riêng bằng `Sys.maxrss` trong tiến trình độc lập (xem Mục Bộ nhớ). Với SPMF, thời gian và bộ nhớ lấy từ log của chương trình Java.

7.2 Tập dữ liệu benchmark

Bảng 10 tóm tắt các tập dữ liệu dùng trong thực nghiệm. Các tập dữ liệu này là benchmark phổ biến trong cộng đồng FIM, lấy từ kho dữ liệu FIMI [Goethals and Zaki \(2003\)](#). Các số liệu trong bảng được tính trực tiếp từ file trong thư mục `data/benchmark`.

Bảng 10: Các tập dữ liệu benchmark dùng trong Mục 7.

Dataset	#Trans.	#Items	AvgLen	Đặc điểm
Chess	3,196	75	37.00	Dày, giao dịch dài
Mushrooms	8,416	119	23.00	Dày, nhiều item đồng xuất hiện
Retail	88,162	16,470	10.31	Thưa, dữ liệu bán lẻ thực
T10I4D100K	100,000	870	10.10	Tổng hợp, thưa
Accidents	340,183	468	33.80	Rất lớn và dày

7.3 Tính đúng đắn

Kết quả trong `correctness.csv` cho thấy mọi cấu hình được kiểm tra đều khớp hoàn toàn với SPMF: `match_ratio = 1.0` và `support_mismatch = 0`. Bảng 11 liệt kê một số điểm đại diện.

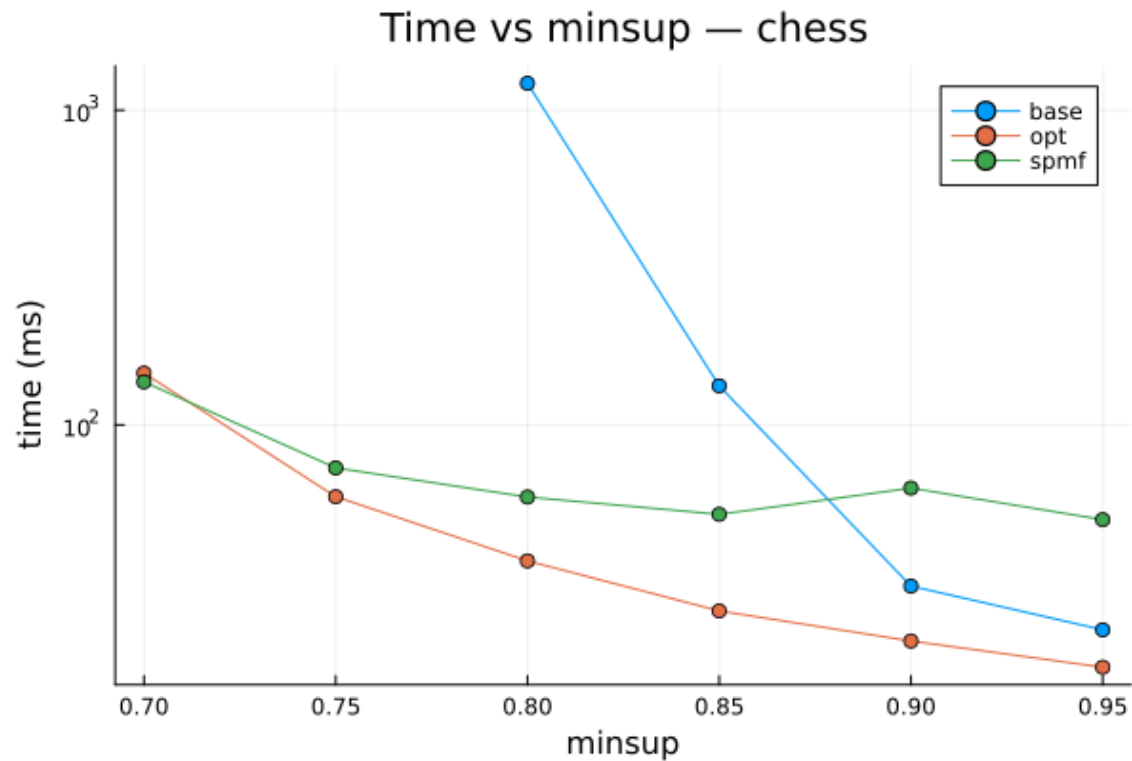
Bảng 11: Kết quả correctness đại diện khi so với SPMF.

Dataset	minsup	Algo	#Ours	#SPMF	Match ratio
Chess	0.80	base/opt	8,227	8,227	1.0
Mushrooms	0.25	base/opt	5,393	5,393	1.0
Retail	0.01	base/opt	159	159	1.0
T10I4D100K	0.01	base/opt	385	385	1.0
Accidents	0.70	opt	529	529	1.0

Kết quả này xác nhận hai điểm. Thứ nhất, parser, cách tính minsup tuyệt đối và support của nhóm tương thích với SPMF trên dữ liệu benchmark. Thứ hai, các tối ưu trong `FPGrowthOpt` không làm thay đổi output so với bản cơ bản.

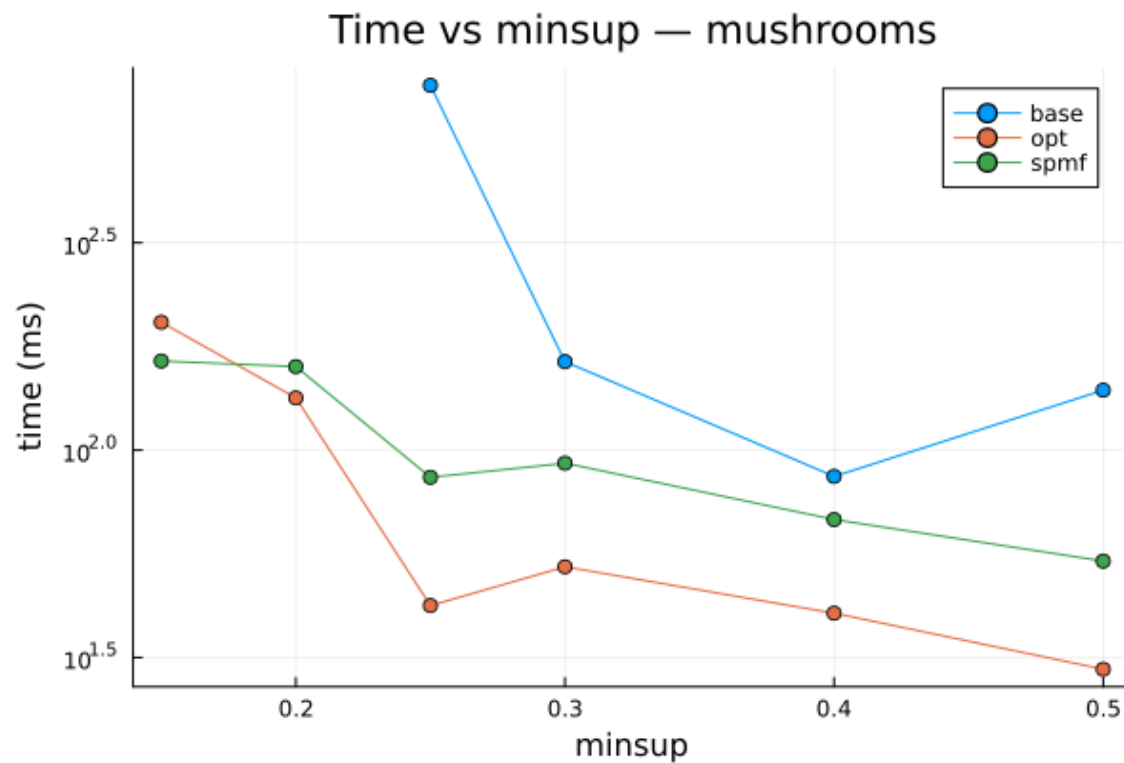
7.4 Thời gian chạy theo minsup

Hình 2 đến Hình 5 cho thấy thời gian chạy khi minsup giảm. Xu hướng chung là minsup càng thấp thì số lượng frequent itemset càng tăng và thời gian chạy tăng theo. Điều này phù hợp với phân tích lý thuyết: khi ngưỡng thấp, nhiều item sống sót qua bước tỉa hơn, conditional tree rộng hơn và output lớn hơn.



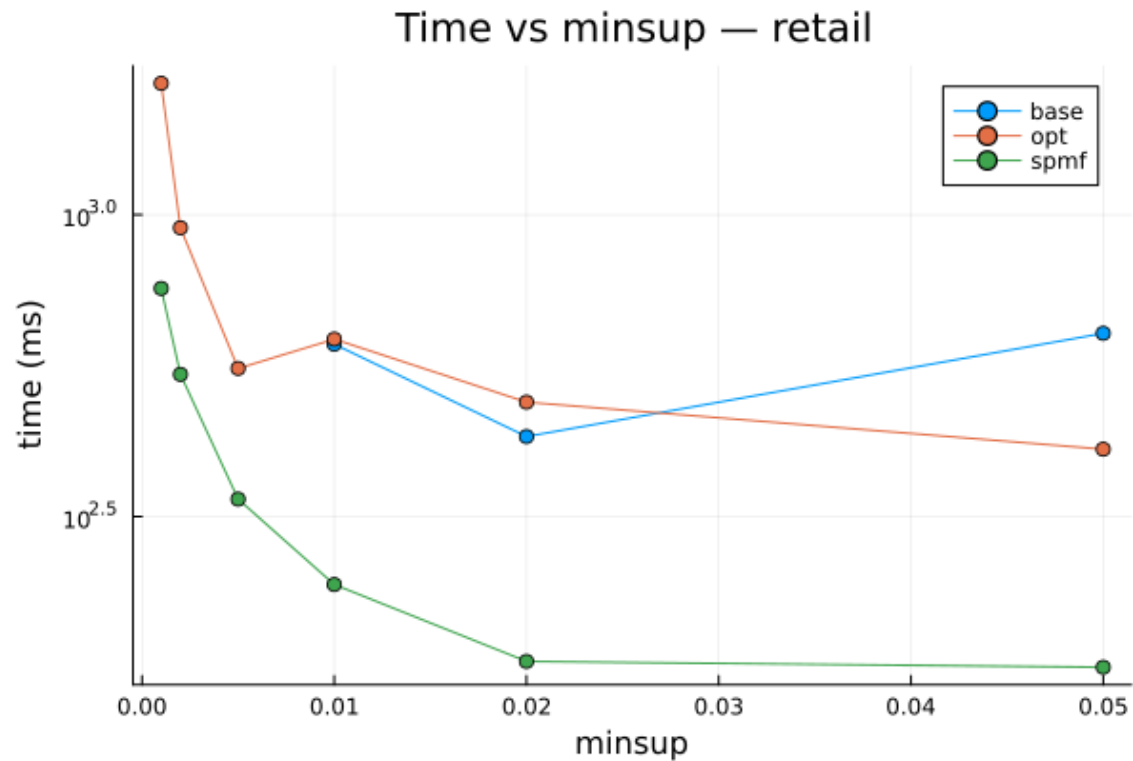
Hình 2: Thời gian chạy theo minsup trên Chess.

Trên Chess, bản tối ưu thể hiện cải thiện rõ ở vùng output lớn. Tại minsup 0.8, **base** mất 1216.93 ms trong khi **opt** mất 36.98 ms, nhanh hơn khoảng 32.91 lần. Khi minsup rất cao như 0.95, output chỉ còn 77 itemsets nên khoảng cách thu hẹp mạnh và lợi thế của bản tối ưu gần như không còn đáng kể.



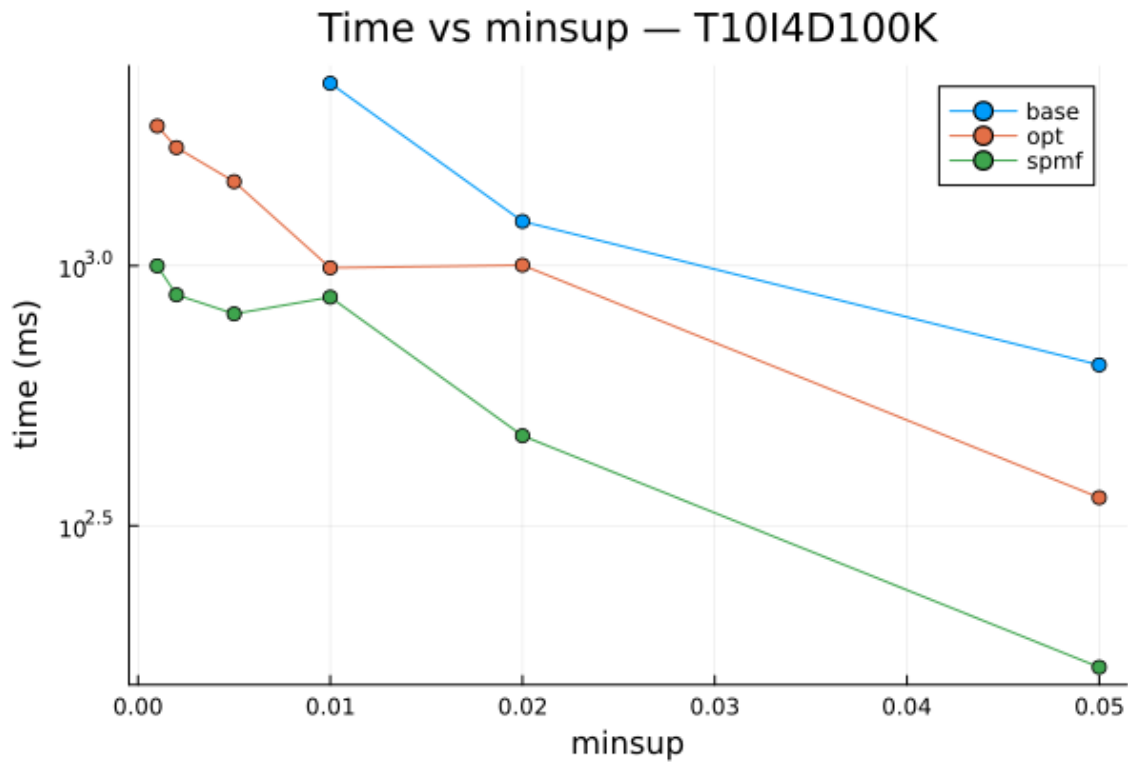
Hình 3: Thời gian chạy theo minsup trên Mushrooms.

Hình 3 thể hiện xu hướng tương ứng trên Mushrooms. Tại minsup 0.25, **opt** nhanh hơn **base** khoảng 17.97 lần. Tuy nhiên ở một số ngưỡng cao hơn, chênh lệch không ổn định do chi phí dựng conditional tree và chi phí JIT có thể chiếm tỷ trọng lớn khi output nhỏ.



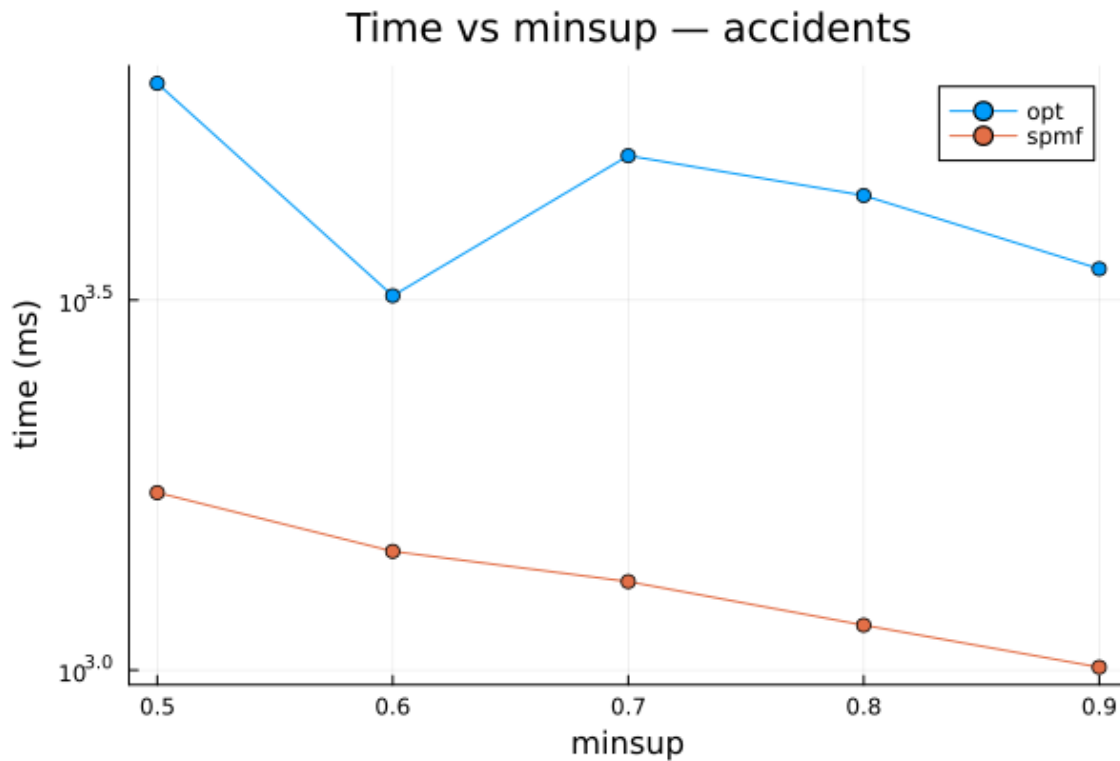
Hình 4: Thời gian chạy theo minsup trên Retail.

Hình 4 cho thấy Retail là dữ liệu thưa nên số itemset đầu ra nhỏ hơn nhiều so với Chess và Mushrooms. Ở minsup 0.01, bản tối ưu và bản cơ bản xấp xỉ nhau (622.16 ms so với 610.27 ms của base). SPMF vẫn nhanh hơn đáng kể, phản ánh khoảng cách giữa cài đặt học thuật của nhóm và thư viện Java đã được tối ưu lâu dài.



Hình 5: Thời gian chạy theo minsup trên T10I4D100K.

Trên T10I4D100K, **opt** nhanh hơn **base** ở các điểm **base** còn chạy được. Tại minsup 0.01, tốc độ cải thiện là khoảng 2.26 lần; tại minsup 0.05, cải thiện khoảng 1.80 lần. Dữ liệu này thừa và tổng hợp nên lợi ích nén tiền tố không mạnh như trên Chess.

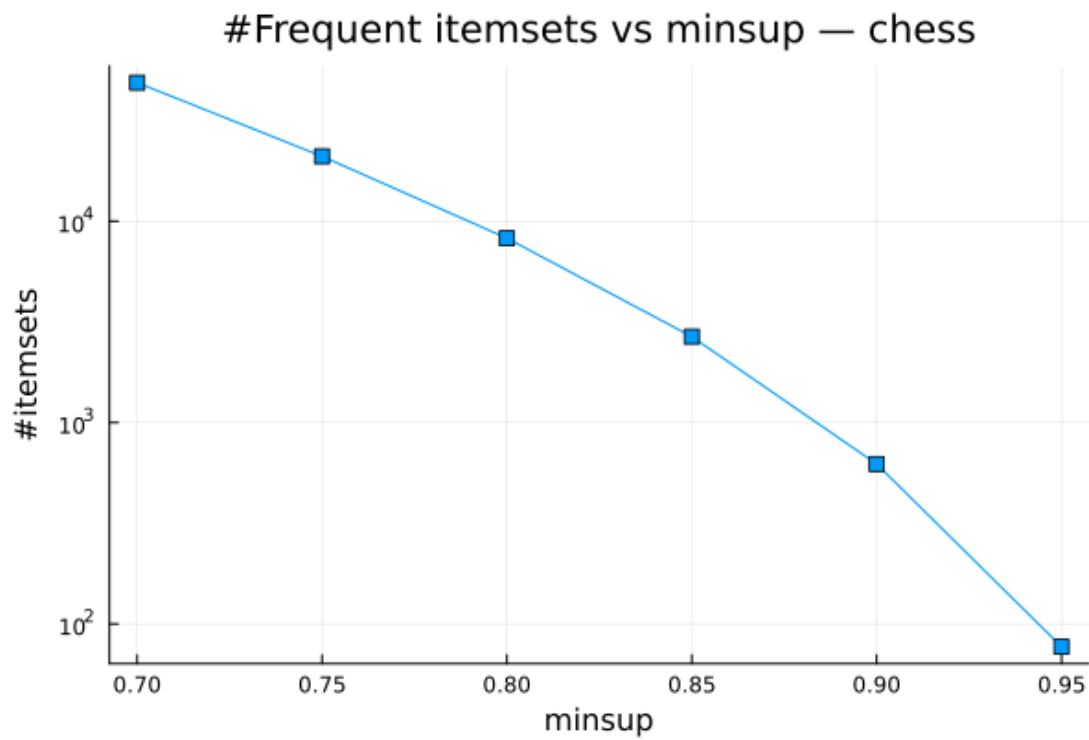


Hình 6: Thời gian chạy theo minsup trên Accidents.

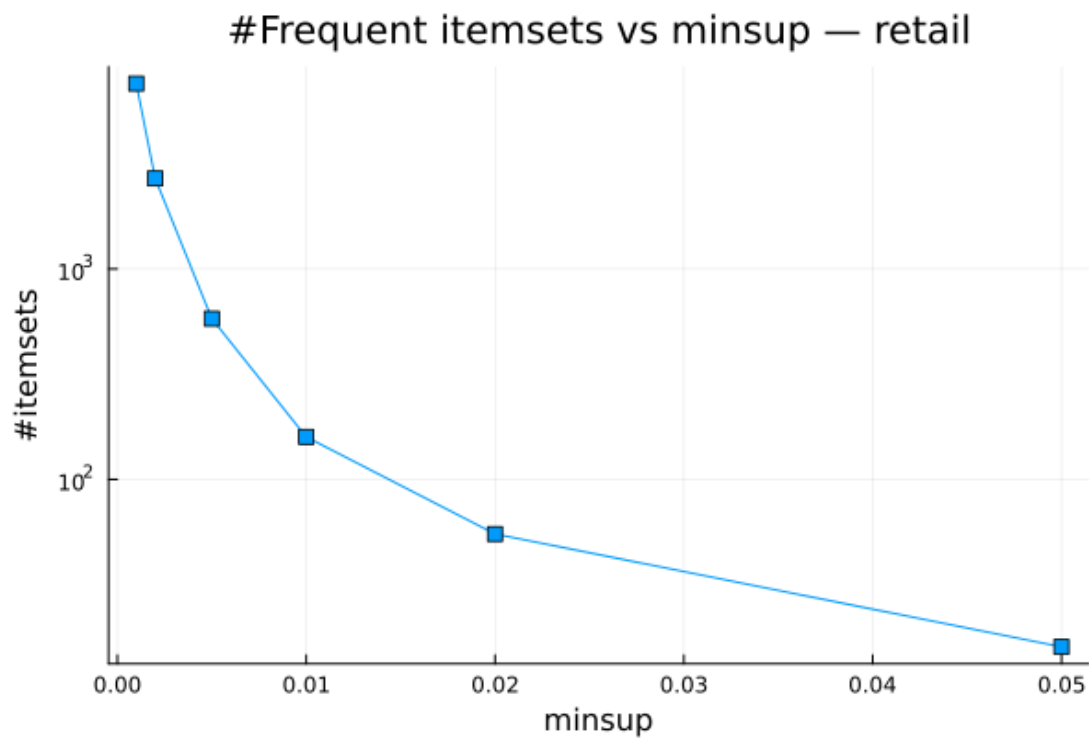
Hình 6 trình bày kết quả trên Accidents. Bản base bị bỏ qua vì dữ liệu vừa lớn vừa dày. Bản tối ưu vẫn chạy được ở các ngưỡng từ 0.9 đến 0.5, nhưng thời gian nằm quanh 3.2–6.2 giây và chậm hơn SPMF khoảng 2.2–3.8 lần. Điểm yếu chính là cài đặt của nhóm vẫn cấp phát nhiều object cho conditional trees và chưa tối ưu quản lý bộ nhớ như SPMF.

7.5 Số lượng frequent itemset theo minsup

Hình 7 và Hình 8 minh họa quan hệ giữa minsup và số itemset đầu ra. Chess tăng từ 77 itemsets ở minsup 0.95 lên 48,731 itemsets ở minsup 0.7. Retail tăng từ 16 itemsets ở minsup 0.05 lên 7,589 itemsets ở minsup 0.001.



Hình 7: Số frequent itemset theo minsup trên Chess.



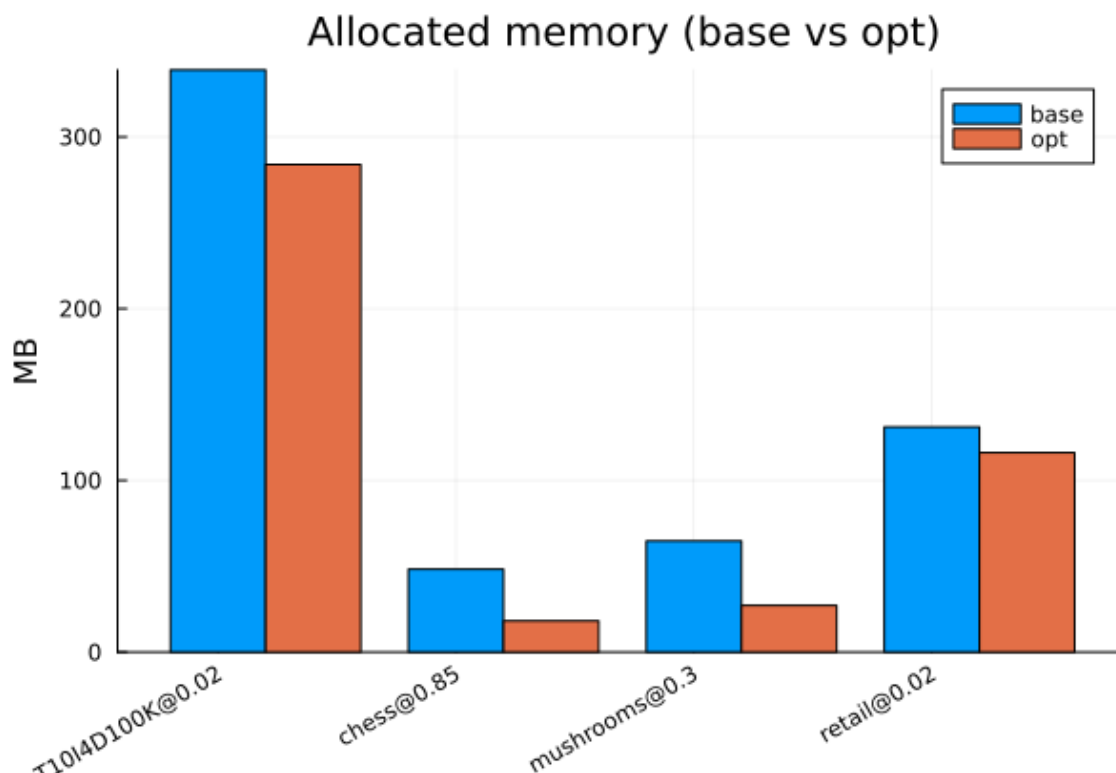
Hình 8: Số frequent itemset theo minsup trên Retail.

Sự khác biệt giữa hai hình cho thấy độ dày dữ liệu ảnh hưởng mạnh đến output size. Chess có giao

dịch dài và nhiều item đồng xuất hiện nên khi minsup giảm, nhiều tổ hợp dài cùng trở nên phổ biến. Retail có nhiều item phân biệt nhưng giao dịch ngắn hơn, vì vậy số mẫu tăng chậm hơn trong cùng khoảng ngưỡng tương đối.

7.6 Bộ nhớ

Hình 9 so sánh lượng byte cấp phát giữa base và opt tại một minsup trung bình của từng dataset. Trên Chess ở minsup 0.85, base cấp phát 48.29 MB còn opt cấp phát 18.19 MB, giảm khoảng 2.65 lần. Trên Mushrooms ở minsup 0.3, mức giảm là 64.72 MB xuống 27.17 MB, tức khoảng 2.38 lần. Với Retail và T10I4D100K, mức giảm nhỏ hơn, lần lượt khoảng 1.13 và 1.19 lần.



Hình 9: Bộ nhớ cấp phát của base và opt tại minsup trung bình.

Lợi ích bộ nhớ đến từ mã hoá item sang số nguyên, dùng Dict cho children và khai thác conditional tree có tĩa sớm. Cột `alloc_bytes` phản ánh tổng lượng cấp phát trong Julia, tức áp lực lên bộ thu gom rác (GC), chứ không phải dung lượng thường trú lớn nhất của tiến trình.

Để đo trực tiếp peak memory thật, nhóm bổ sung một thực nghiệm riêng (`experiments/measure_memory.jl`): mỗi cấu hình được chạy trong một tiến trình Julia độc lập rồi đọc `Sys.maxrss()` — đỉnh resident set size (RSS) của tiến trình. Vì mỗi tiến trình mới reset RSS, giá trị `maxrss` cuối là peak của đúng

lần chạy đó. Hàng *sàn* chỉ load package và dữ liệu mà không mining, cho biết phần RSS do runtime Julia chiếm. Bảng 12 tổng hợp kết quả ở các điểm tương ứng với Hình 9.

Bảng 12: Peak RSS thật (`Sys.maxrss`) đo trong tiến trình riêng cho mỗi lần chạy. Cột *sàn* là cấu hình chỉ load dữ liệu, không mining.

Dataset	minsup	Sàn (MB)	base peak (MB)	opt peak (MB)
Chess	0.80	401.55	425.74	409.83
Chess	0.85	401.55	420.17	402.07
Mushrooms	0.30	404.57	430.71	415.12
Retail	0.01	446.30	509.08	450.73
T10I4D100K	0.01	439.71	771.17	772.19

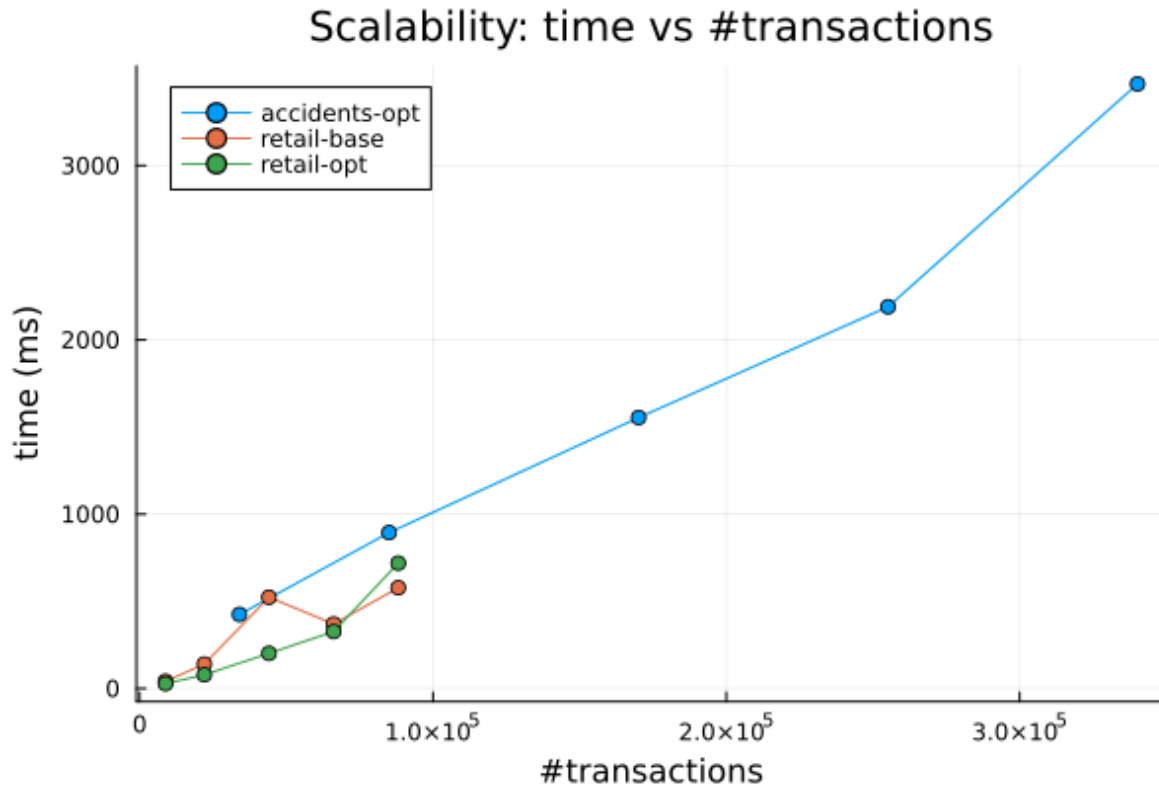
Bảng 12 cho thấy ba điểm. Thứ nhất, sàn RSS do runtime Julia, load package và dữ liệu chiếm khoảng 400–446 MB, lớn hơn nhiều so với phần bộ nhớ làm việc của thuật toán, nên peak RSS tuyệt đối bị chi phối bởi sàn này. Thứ hai, sau khi trừ sàn, bản tối ưu thường trú ít hơn bản cơ bản trên dữ liệu dày: trên Retail ở minsup 0.01 peak giảm từ 509.08 MB (base) xuống 450.73 MB (opt), và trên Chess 0.80 cũng như Mushrooms 0.30 opt đều thấp hơn base, nhất quán với xu hướng của `alloc_bytes`. Thứ ba, peak RSS thật nhỏ hơn nhiều so với tổng `alloc_bytes`: ví dụ Chess 0.80 có `alloc_bytes` của base khoảng 360 MB (Bảng 9) nhưng peak RSS chỉ cao hơn sàn khoảng 24 MB, vì phần lớn cấp phát là tạm thời và bị GC thu hồi nên không cùng lúc thường trú. Hai chỉ số do đó bổ sung cho nhau: `alloc_bytes` đo áp lực cấp phát/GC, còn peak RSS đo dung lượng thực tế của tiến trình. Riêng T10I4D100K có output nhỏ (385 itemsets) nên peak của base và opt xấp xỉ nhau và nằm trong vùng nhiễu, ở đó FP-tree thừa của 100,000 giao dịch là thành phần chiếm bộ nhớ chính.

7.7 Khả năng mở rộng

Hình 10 trình bày thực nghiệm scalability, lấy các prefix 10%, 25%, 50%, 75% và 100% của Retail và Accidents. Với Retail ở minsup 0.01, opt tăng từ 27.70 ms trên 8,816 giao dịch lên 718.77 ms trên 88,162 giao dịch. Bản opt nhanh hơn base ở hầu hết các kích thước Retail, rõ nhất tại 25% dữ liệu: 78.63 ms so với 139.96 ms; ở mức 100% hai bản xấp xỉ nhau do output nhỏ và nhiễu JIT/GC chi phối.

Với Accidents ở minsup 0.7, opt tăng từ 424.78 ms trên 34,018 giao dịch lên 3467.97 ms trên 340,183 giao dịch. Xu hướng tăng không hoàn toàn tuyến tính vì số itemset đầu ra dao động quanh 522–554,

nhưng chi phí dựng và duyệt conditional tree vẫn tăng theo số giao dịch.



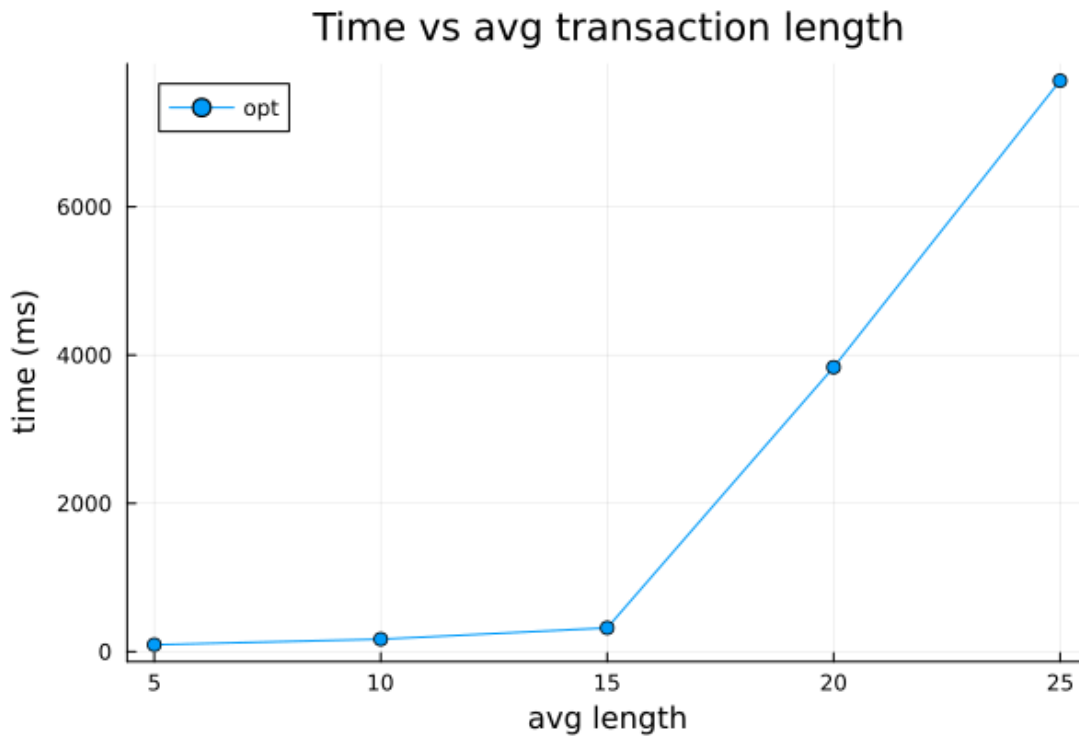
Hình 10: Khả năng mở rộng theo số lượng giao dịch.

7.8 Ảnh hưởng của độ dài giao dịch

Thí nghiệm cuối dùng dữ liệu tổng hợp 20,000 giao dịch, 100 item, minsup 0.03, và tăng độ dài giao dịch trung bình từ 5 đến 25. Bảng 13 cho thấy thời gian tăng mạnh khi độ dài trung bình đạt 20 và 25.

Bảng 13: Ảnh hưởng của độ dài giao dịch trung bình lên thời gian chạy.

AvgLen	Time ms	#Itemsets
5	93.46	100
10	168.26	100
15	320.86	100
20	3836.42	5,050
25	7701.39	5,050



Hình 11: Thời gian chạy khi tăng độ dài giao dịch trung bình.

Kết quả này phù hợp với lý thuyết: giao dịch dài làm tăng xác suất các item cùng xuất hiện, làm FP-tree dày hơn và số mẫu phổ biến tăng. Khi output size nhảy từ 100 lên 5,050 itemsets, thời gian cũng tăng theo bậc lớn hơn nhiều so với mức tăng tuyến tính của độ dài giao dịch; Hình 11 minh hoạ rõ bước nhảy này.

7.9 Nhận xét và hướng tối ưu tiếp theo

Thực nghiệm cho thấy bản tối ưu của nhóm đúng so với SPMF và cải thiện rõ so với bản cơ bản ở các vùng có output lớn. Lợi ích nổi bật nhất xuất hiện trên dữ liệu dày như Chess và Mushrooms, nơi cấu trúc FP-tree và single-path shortcut phát huy tác dụng. Trên dữ liệu thưa như Retail và T10I4D100K, cải thiện vẫn có nhưng nhỏ hơn vì các tiền tố chung ít hơn.

Điểm yếu chính là bản tối ưu vẫn chậm hơn SPMF trên phần lớn cấu hình lớn. Hai hướng tối ưu cụ thể có thể làm tiếp là: thứ nhất, giảm cấp phát khi dựng conditional tree bằng cách tái sử dụng buffer/vector và tránh tạo nhiều **Vector** tạm; thứ hai, thay header dạng Dict bằng cấu trúc vector theo mã item liên tục để giảm overhead hash lookup. Ngoài ra, có thể nghiên cứu biểu diễn transaction bằng bitset cho bước tính support hoặc song song hoá các nhánh conditional tree độc

lập.

8 Ứng dụng

8.1 Bài toán phân tích giỏ hàng

Ứng dụng được chọn là phân tích giỏ hàng (Market Basket Analysis). Với mỗi giao dịch là một hoá đơn mua hàng, frequent itemsets cho biết các nhóm sản phẩm thường xuất hiện cùng nhau. Từ các itemset phổ biến, ta sinh luật kết hợp dạng $X \Rightarrow Y$, trong đó X là vế trái, Y là vế phải và $X \cap Y = \emptyset$.

Ba chỉ số được dùng để đánh giá luật là support, confidence và lift:

$$\text{conf}(X \Rightarrow Y) = \frac{\text{sup}(X \cup Y)}{\text{sup}(X)}$$

và

$$\text{lift}(X \Rightarrow Y) = \frac{\text{conf}(X \Rightarrow Y)}{\text{sup}_{rel}(Y)}.$$

Confidence đo xác suất mua Y khi đã mua X . Lift đo mức tăng so với xác suất mua Y độc lập [Brin et al. \(1997\)](#). Nếu lift lớn hơn 1, X và Y có xu hướng xuất hiện cùng nhau nhiều hơn kỳ vọng ngẫu nhiên [Agrawal et al. \(1993\)](#).

8.2 Dữ liệu và thiết lập

Nhóm sử dụng tập `data/groceries.txt`, gồm 9,835 giao dịch và 169 mặt hàng. Độ dài giao dịch trung bình là 4.41 item, giao dịch dài nhất có 32 item. Vì tên mặt hàng trong Groceries có thể chứa khoảng trắng, dữ liệu được tách theo dấu phẩy trước khi đưa vào thuật toán FP-Growth của nhóm. Thiết lập khai thác là:

- $\text{minsup} = 0.01$, tương đương ít nhất $\lceil 0.01 \times 9835 \rceil = 99$ giao dịch.
- $\text{minconf} = 0.2$.
- Luật được lọc theo $\text{lift} > 1$ rồi sắp theo lift giảm dần; nếu lift bằng nhau thì sắp theo thứ tự bảng chữ cái của vế trái, sau đó của vế phải.

Với thiết lập này, thuật toán tìm được 333 frequent itemsets và sinh 231 luật thoả minconf và có lift lớn hơn 1.

8.3 Top-10 luật theo lift

Bảng 14 trình bày 10 luật có lift cao nhất. Support trong bảng là support tương đối.

Bảng 14: Top-10 luật kết hợp trên Groceries theo lift, với minsup = 0.01 và minconf = 0.2.

Vế trái X	Vế phải Y	Support	Confidence	Lift
citrus fruit + other vegetables	root vegetables	0.0104	0.3592	3.2950
other vegetables + yogurt	whipped/sour cream	0.0102	0.2342	3.2671
other vegetables + tropical fruit	root vegetables	0.0123	0.3428	3.1448
beef	root vegetables	0.0174	0.3314	3.0404
citrus fruit + root vegetables	other vegetables	0.0104	0.5862	3.0296
root vegetables + tropical fruit	other vegetables	0.0123	0.5845	3.0210
other vegetables + whole milk	root vegetables	0.0232	0.3098	2.8421
root vegetables	other vegetables + whole milk	0.0232	0.2127	2.8421
butter	other vegetables + whole milk	0.0115	0.2073	2.7706
curd + whole milk	yogurt	0.0101	0.3852	2.7614

8.4 Diễn giải kinh doanh

Các luật mạnh nhất tập trung quanh nhóm rau củ, sữa và sữa chua. Luật “citrus fruit + other vegetables $\Rightarrow$ root vegetables” có lift 3.2950, nghĩa là trong các giỏ hàng đã có trái cây họ cam chanh và rau khác, khả năng xuất hiện root vegetables cao hơn khoảng 3.3 lần so với tần suất nền của root vegetables. Đây là tín hiệu phù hợp cho gợi ý mua kèm hoặc bố trí kệ gần nhau giữa nhóm rau củ và trái cây tươi.

Luật “beef $\Rightarrow$ root vegetables” có confidence 0.3314 và lift 3.0404. Về mặt kinh doanh, người mua thịt bò có xu hướng mua thêm rau củ dùng cho món hầm, súp hoặc bữa ăn chính. Siêu thị có thể dùng luật này để thiết kế combo nguyên liệu hoặc coupon giảm giá chéo.

Nhóm luật liên quan tới *whole milk*, *curd* và *yogurt* cho thấy các sản phẩm sữa có quan hệ đồng mua rõ. Ví dụ “curd + whole milk $\Rightarrow$ yogurt” có confidence 0.3852 và lift 2.7614. Luật này phù

hợp cho gợi ý trong giỏ hàng trực tuyến: khi khách đã chọn curd và whole milk, hệ thống có thể đề xuất yogurt như một sản phẩm bổ sung.

Vì ngưỡng minconf đặt thấp (0.2), top theo lift còn chứa một số luật có confidence khiêm tốn, ví dụ “other vegetables + yogurt $\Rightarrow$ whipped/sour cream” chỉ có confidence 0.2342 dù lift đạt 3.2671. Lift cao trong trường hợp này phản ánh whipped/sour cream là mặt hàng hiếm, nên một mức tăng nhỏ về xác suất đã đẩy lift lên. Ngoài ra một vài luật có vế phải gồm hai item, như “root vegetables $\Rightarrow$ other vegetables + whole milk”. Khi đọc các luật này cần xét confidence cùng lift, không chỉ riêng lift.

Một điểm cần lưu ý là support của các luật top lift chỉ quanh 1–2.3%. Các luật này có ý nghĩa định hướng gợi ý theo ngữ cảnh, nhưng không nên xem là luật bao phủ phần lớn khách hàng. Nếu mục tiêu là khuyến mãi diện rộng, cần cân bằng lift với support tuyệt đối và lợi nhuận của từng sản phẩm.

9 Kết luận

Đồ án đã hoàn thành việc nghiên cứu, cài đặt và đánh giá họ thuật toán FP-Growth cho bài toán khai thác tập phổ biến. Về lý thuyết, báo cáo trình bày các định nghĩa cốt lõi của FIM, tính chất Apriori, cấu trúc FP-tree, cơ chế conditional pattern base, trường hợp single-path và biến thể FP-Max. Hai ví dụ tay cho thấy thuật toán hoạt động đúng trên cả trường hợp cơ sở lẫn tình huống đặc biệt.

Về cài đặt, nhóm xây dựng package Julia `FrequentItemsetMining` với ba nhánh: FP-Growth cơ bản, FP-Growth tối ưu và FP-Max. Bản tối ưu sử dụng mã hoá item sang số nguyên, node con dạng `Dict`, khai thác conditional FP-tree đệ quy, tĩa sớm và shortcut single-path. Output được giữ tương thích với SPMF để thuận tiện kiểm thử và đối chiếu.

Về thực nghiệm, kết quả correctness đạt `match_ratio = 1.0` và `support_mismatch = 0` trên các cấu hình benchmark được kiểm tra. Bản tối ưu cải thiện rõ so với bản cơ bản trên dữ liệu dày và output lớn, ví dụ nhanh hơn khoảng 32.91 lần trên Chess tại minsup 0.8 và khoảng 17.97 lần trên Mushrooms tại minsup 0.25. Tuy vậy, bản tối ưu vẫn chậm hơn SPMF ở nhiều cấu hình lớn, chủ yếu do chi phí cấp phát conditional tree và overhead cấu trúc dữ liệu trong cài đặt học thuật.

Phần ứng dụng trên Groceries cho thấy thuật toán có thể sinh các luật kết hợp có ý nghĩa thực tế, chẳng hạn các luật liên quan tới *root vegetables*, *other vegetables*, *whole milk* và *yogurt*. Các luật

này có thể hỗ trợ gợi ý sản phẩm, bố trí kệ hàng và thiết kế combo.

Các hạn chế còn lại của đề án gồm: bản base phải skip một số cấu hình do nguy cơ bùng nổ; và bản tối ưu chưa khai thác song song các nhánh conditional tree độc lập. Trong tương lai, nhóm có thể giảm cấp phát bằng buffer tái sử dụng, thay Dict header bằng vector theo mã item liên tục, và song song hoá các nhánh conditional tree độc lập.

Tài liệu

- Agrawal, R., Imielinski, T., and Swami, A. (1993). Mining association rules between sets of items in large databases. In *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*, pages 207–216.
- Agrawal, R. and Srikant, R. (1994). Fast algorithms for mining association rules in large databases. In *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*, pages 487–499.
- Brin, S., Motwani, R., Ullman, J. D., and Tsur, S. (1997). Dynamic itemset counting and implication rules for market basket data. In *Proceedings of the 1997 ACM SIGMOD International Conference on Management of Data*, pages 255–264.
- Fournier-Viger, P., Gomariz, A., Gueniche, T., Soltani, A., Wu, C.-W., and Tseng, V. S. (2014). SPMF: A java open-source pattern mining library. *Journal of Machine Learning Research*, 15:3389–3393.
- Goethals, B. and Zaki, M. J. (2003). Frequent itemset mining dataset repository. FIMI Workshop, <http://fimi.uantwerpen.be/data/>.
- Grahne, G. and Zhu, J. (2003). High performance mining of maximal frequent itemsets. In *Proceedings of the Workshop on Frequent Itemset Mining Implementations*.
- Han, J., Pei, J., and Yin, Y. (2000). Mining frequent patterns without candidate generation. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pages 1–12.
- Zaki, M. J. (2000). Scalable algorithms for association mining. *IEEE Transactions on Knowledge and Data Engineering*, 12(3):372–390.

A Phụ lục: Hướng dẫn tái lập

Phụ lục này tóm tắt các bước tái lập toàn bộ mã nguồn, kiểm thử và thực nghiệm của đồ án. Nội dung tương ứng với tệp `README.md` trong kho mã nguồn; mọi lệnh đều chạy từ thư mục gốc của repo. Mã nguồn đầy đủ tại github.com/pineapplechoco/dma-lab2.

A.1 Cấu trúc thư mục

- `src/`: package Julia `FrequentItemsetMining` (FP-Growth cơ bản, FP-Growth tối ưu, FP-Max, sinh luật kết hợp, I/O).
- `main.jl`: giao diện dòng lệnh.
- `test/`: bộ kiểm thử correctness và benchmark.
- `experiments/`: harness thực nghiệm Chương 7 và ứng dụng Chương 8.
- `data/`: CSDL toy, benchmark và Groceries.
- `docs/Report.pdf`: báo cáo.
- `Project.toml`, `Manifest.toml`: khai báo và khoá phiên bản phụ thuộc.

Đồ án không sử dụng bất kỳ thư viện FIM có sẵn nào; SPMF chỉ được gọi như một công cụ hộp đen để đối chiếu kết quả ở Chương 7.

A.2 Môi trường và cài đặt

Yêu cầu Julia ≥ 1.9 (đã kiểm thử trên Julia 1.12). Cách khuyến nghị là dùng `juliaup` — trình quản lý phiên bản chính thức của Julia:

```
1 curl -fsSL https://install.julialang.org | sh
2 juliaup add 1.12          # optional: pin the tested version
3 julia --version           # expected: julia version 1.12.x (or >= 1.9)
```

Listing 6: Cài Julia bằng `juliaup` (Linux/macOS) và ghim phiên bản.

Trên Windows có thể cài từ Microsoft Store (tìm “Julia”) hoặc chạy `winget install julia -s msstore`. Sau khi có Julia, cài phụ thuộc theo `Manifest.toml` đã khoá để đảm bảo tái lập:


```
1 julia --project=. -e 'using Pkg; Pkg.instantiate()'
```

Listing 7: Cài đặt phụ thuộc.

A.3 Chạy chương trình

Giao diện dòng lệnh nhận đường dẫn dữ liệu, ngưỡng minsup tương đối, thuật toán và đường dẫn output (tùy chọn):

```
1 julia --project=. main.jl <input_path> <minsup> [fp-growth|fp-growth-opt|fp-max|  
  rules] [output_path]
```

Listing 8: Cú pháp dòng lệnh.

Ví dụ khai thác tập phổ biến trên CSDL toy với minsup 0.6:

```
1 julia --project=. main.jl data/toy/test_1.txt 0.6 fp-growth-opt output.txt
```

Listing 9: Khai thác itemset.

Đầu vào theo định dạng SPMF (mỗi giao dịch một dòng, item cách nhau bằng khoảng trắng; parser cũng chấp nhận dòng có dấu phẩy). Đầu ra theo dạng `item1 item2 ... #SUP: support_count`. Chế độ `rules` sinh trực tiếp luật kết hợp ($\text{lift} > 1$, sắp theo lift giảm dần) từ tập phổ biến. Ngưỡng confidence đặt qua biến môi trường `MINCONF` (mặc định 0.2), số luật in ra qua `TOPK` (mặc định 10). Với dữ liệu mà tên item chứa khoảng trắng (ví dụ Groceries), đặt `SEP=comma` để parser chỉ tách theo dấu phẩy:

```
1 SEP=comma julia --project=. main.jl data/groceries.txt 0.01 rules rules.csv
```

Listing 10: Sinh luật kết hợp trên Groceries.

Lệnh trên tái tạo đúng kết quả phần ứng dụng: 333 frequent itemsets và 231 luật có lift lớn hơn 1.

A.4 Kiểm thử

```
1 julia --project=. test/runtests.jl
```

Listing 11: Chạy bộ kiểm thử tự động.

Bộ kiểm thử gồm: kiểm tra parser ba định dạng; FP-Growth và FP-Max trên CSDL toy ở Chương 5; đối chiếu bản tối ưu với bản cơ bản trên năm CSDL (hai toy và ba benchmark: chess, mushrooms,

retail); kiểm tra sinh luật kết hợp; và benchmark đối chiếu base/opt trên chess. Các tỉ lệ tốc độ/bộ nhớ in trong dòng [bench] là số đo on-the-fly, phụ thuộc máy và lần chạy (JIT warm-up, GC) nên có thể dao động, không nhất thiết trùng với số liệu cố định trong báo cáo.

A.5 Tái lập thực nghiệm Chương 4

Thực nghiệm cần SPMF (làm chuẩn đối chiếu, yêu cầu Java ≥ 8) và đầy đủ năm tập benchmark.

1. Tải SPMF jar (bị .gitignore vì nặng):

```
1 bash experiments/download_spmf.sh
```

2. Tải dataset. Bốn tập chess, mushrooms, retail, T10I4D100K đã có sẵn trong data/benchmark/; bổ sung accidents.txt (> 25 MB, không đưa vào repo) từ Google Drive vào data/benchmark/:

[Thư mục dữ liệu trên Google Drive.](#)

3. Chạy thực nghiệm (sinh các file CSV trong experiments/results/):

```
1 julia --project=. experiments/run_experiments.jl
```

Có thể giới hạn tập dữ liệu qua biến môi trường, ví dụ bỏ qua accidents:

```
1 EXP_DATASETS=chess,mushrooms,retail,T10I4D100K julia --project=. experiments/  
run_experiments.jl
```

4. Đo peak RSS thật (sinh experiments/results/memory.csv):

```
1 julia --project=. experiments/measure_memory.jl
```

5. Sinh biểu đồ (vào experiments/figures/):

```
1 julia --project=. experiments/make_figures.jl
```

A.6 Tái lập ứng dụng Chương 5

Sinh luật kết hợp trên tập Groceries (minsup 0.01, minconf 0.2), in top-10 luật theo lift và ghi experiments/results/rules_groceries.csv:

```
1 julia --project=. experiments/application.jl
```

Listing 12: Phân tích giỏ hàng.

Kết quả mong đợi: 333 frequent itemsets và 231 luật có lift lớn hơn 1. Kết quả này cũng đạt được trực tiếp qua chế độ `rules` của `main.jl` như ở mục Chạy chương trình.

A.7 Tính tái lập

Mọi dữ liệu tổng hợp dùng seed cố định (`experiments/datagen.jl`, seed 42); `Manifest.toml` khoá toàn bộ phiên bản phụ thuộc. Số đếm (`#frequent itemsets`, `#luật`) và kết quả correctness là tất định nên chạy lại cho ra cùng giá trị; riêng các số đo thời gian và peak RSS có thể dao động theo máy và lần chạy do JIT, GC và tải hệ thống.